KORREKTURBOGEN Vorlesung Programmieren SpeedyQuiz	DHBW Duale Hochschule Baden-Württemberg
Hilfsmittel: Vorlesungsfolien, Übungsblätter + Aufgaben	**Bearbeitungszeit:** 120 Minuten

Bewertung:

		Kriterium	Mögl. Punkte	Erreichte Punkte
Test	t1	TEST 1: SpeedyQuiz starten 2 Fenster öffnen sich (2 fürs erste, 3 fürs zweite)	5	
	t2	TEST 2: Fenster schließen Beide Fenster schließen sich (2 je Fenster), Programm beendet (1)	5	
	t3	TEST 3: Einen Antwort-Button in einem der Fenster drücken Status-Aktualisierung (muss nicht der korrekte Status sein) und Anzeige neuer Frage im gewählten Fenster (2) und im anderen Fenster (3)	5	
	t4	TEST 4: Countdown abwarten Countdown für Frage zählt runter (1), wenn Countdown abgelaufen: Statusupdate für abgelaufene Frage (1 je Fenster), neue Frage (1 je Fenster)	5	
	t5	TEST 5: Zu Ende Spielen Zuende Spielen möglich (2), Dialog für Auswertung (unabhängig vom Inhalt) erscheint (3)	5	
a	a1	Klasse Question Klasse korrekt angelegt (1), mit 3 Attributen(Fragentext, Antworten, Index) (3), Konstruktor laut Vorgabe (1)	5	
b	b1	Interface GameClient Interface (keine Klasse!) korrekt angelegt (1)	1	
	b2	Interface-Methoden Alle 6 Methoden korrekt angelegt (die drei argumentlosen Methoden insg. 2, die drei Methoden mit Argumenten je 1)	5	
	b3	Komplexer Aufzählungstyp Status Komplexe enum angelegt (1), mit beiden Attributen (Farbe, Punkte) (2), beide privat mit Getter+Setter (1)	4	
c	c1	Klasse Game Korrekt angelegt und Konstruktor nimmt Fragenkatalog und Anzahl entgegen (1), zufällige Auswahl der Fragen mit richtiger Anzahl (1), ohne Duplikate (1)	3	
	c2	Klasse GameException Klasse korrekt angelegt die von Exception erbt (1), Konstruktor für Nachricht (1), wird mit richtiger Nachricht bei zu großer Anzahl geworfen (1)	3	
	c3	Methode registerClient(GameClient) Korrekt angelegt (1) mit Speicherung in geeigneter Datenstruktur (1), Aufnehmen des Clients nur, wenn Spiel noch nicht gestartet (1)	3	
	c4	Methode getQuestionCount() Korrekt angelegt und gibt Anzahl der Fragen zurück (1)	1	
	c5	Methode startGame() Korrekt angelegt mit Schleife, die erste Frage verteilt (1)	1	
	c6	Methode answerSelected(GameClient,int) Korrekt angelegt (1), sendendem Client wird CORRECT/WRONG gesetzt (1) allen andern NO_ANSWER (1)	3	
	c7	Zustandsupdate nach Antwort Verteilung der nächsten Frage an alle Clients, bzw Ende des Spiels (1)	1	
d	d1	Klasse GameTerm Korrekt (inkl. implements GameClient!) angelegt (1), korrekter Konstruktor (1), Speicherung in Instanzvariablen (1)	3	
	d2	Benutzeroberfläche Titelzeile (1), Labels für Fragen werden oben verteilt (1), Fragenbereich (1), Label für Countdown (1), 2x2 Antwort-Button-Matrix (1)	5	
	d3	Klasse QuestionNumberLabel Korrekt angelegt (erbt von JLabel) (1), Attribut für Status (1), Update der Farbe bei Zustandsänderung (1)	3	
	d4	Button-Action ActionListener angemeldet (1), übermittelt Game-Instanz den aktuellen GameClient und selektierten Index via Game#answerSelected (insg. 2)	3	
	d5	Implementierung von gameIsOver() Deaktiviert alle Buttons (1)	1	
e	e1	Einlesen der Datei Mitgelieferte Datei questions.txt zum Lesen öffnen (1) und zeilenweise Verarbeiten (1)	2	
	e2	Umwandeln zu Question-Objekt Verwendung der bereitgestellten Methode parseQuestion (2)	2	

	e3	Exception-Handling (Dialog) Abfangen sämtlicher Exceptions (1), Anzeige des Dialogs mit gegebenem Titel (1) und Nachricht der Exception (1) sowie Error-Icon (1)	4	
	e4	Ressource schließen Ressource wird sicher geschlossen (2)	2	
f	f1	Highscore-Dialog Dialog mit Dauer des Spiels seit Aufruf von startGame (2), Name+Punkte aller Spieler (4)	6	
	f2	Highscore-Datei Datei highscore.txt wird zum Schreiben geöffnet (1), mit append (1), Dialog-Zeile wird hineingeschrieben (1), Ressource sicher geschlossen (2)	4	
g	g1	Countdown starten Thread wird gestartet (1), sekundenweise von 10 runterzählen (1)	2	
	g2	Sekündliche Aktualisierung der Clients jede Sekunde wird allen Clients mittels setRemainingSeconds(int) die Restzeit mitgeteilt (2)	2	
	g3	Spieler gibt eine Antwort Thread endet wenn ein Spieler eine Antwort gegeben hat (2)	2	
	g4	Countdown abgelaufen Thread ended (1) und an alle Clients wird NO_ANSWER verteilt (1)	2	
	g5	Verteilen der nächsten Frage Die nächste Frage wird gespielt bzw. das Spiel beendet (2)	2	
		Summe	**100**	

Notenskala:

100: 1,0	99: 1,0	98: 1,0	97: 1,0	96: 1,0	95: 1,0	94: 1,0	93: 1,0
92: 1,1	91: 1,1	90: 1,2	89: 1,3	88: 1,3	87: 1,4	86: 1,5	85: 1,5
84: 1,6	83: 1,7	82: 1,7	81: 1,8	80: 1,8	79: 1,9	78: 2,0	77: 2,0
76: 2,1	75: 2,2	74: 2,2	73: 2,3	72: 2,4	71: 2,4	70: 2,5	69: 2,5
68: 2,6	67: 2,7	66: 2,7	65: 2,8	64: 2,9	63: 2,9	62: 3,0	61: 3,1
60: 3,1	59: 3,2	58: 3,2	57: 3,3	56: 3,4	55: 3,4	54: 3,5	53: 3,6
52: 3,6	51: 3,7	50: 3,8	49: 3,8	48: 3,9	47: 4,0	46: 4,0	45: 4,1
44: 4,1	43: 4,2	42: 4,3	41: 4,3	40: 4,4	39: 4,5	38: 4,5	37: 4,6
36: 4,7	35: 4,7	34: 4,8	33: 4,8	32: 4,9	<=31: 5,0		